Лекция 3. Введение в машинное обучение и модуль sklearn

Введение

Машинное обучение (Machine Learning, ML) — это раздел искусственного интеллекта, который изучает методы построения моделей, способных обучаться на данных и делать прогнозы без явного программирования правил [1]. В отличие от традиционного программирования, где разработчик вручную задаёт алгоритм решения задачи, в машинном обучении модель сама находит закономерности в данных[3]. Это делает ML мощным инструментом для решения задач, где правила слишком сложны или неизвестны: распознавание образов, прогнозирование, классификация, рекомендации.

Машинное обучение стало массовым благодаря трём факторам: рост объёмов данных, увеличение вычислительных мощностей и развитие алгоритмов[4]. Сегодня ML применяется в медицине, финансах, маркетинге, транспорте, науке. В курсе «Анализ данных в Python» мы рассматриваем основы ML и библиотеку sklearn — основной инструмент для построения моделей на Python [2].

Постановка задач машинного обучения

Задача машинного обучения формулируется через несколько ключевых понятий. Пространство признаков X — это множество всех возможных описаний объектов. Каждый объект представляется вектором признаков $x = [x_1, x_2, \ldots, x_n]$, где n — размерность пространства. Признаки могут быть числовыми, категориальными, бинарными. Пространство меток Y — это множество возможных ответов. В зависимости от задачи Y может быть непрерывным (регрессия) или дискретным (классификация). Модель — это

функция f: X → Y, которая отображает объект в ответ. Обучение — это процесс поиска такой функции f, которая минимизирует ошибку на данных.

Формально задача обучения с учителем формулируется так: дана размеченная выборка $D = \{(x^i, y^i)\}$, $i = 1, \ldots, m$, где x^i — объект, y^i — истинная метка. Требуется найти функцию f, которая хорошо обобщает на новые объекты, то есть даёт правильные ответы не только на обучающих данных, но и на новых. Качество модели оценивается на тестовой выборке, которая не использовалась при обучении.

Основные типы задач машинного обучения: обучение с учителем (supervised learning), обучение без учителя (unsupervised learning), обучение с подкреплением (reinforcement learning). В обучении с учителем есть размеченные данные, и модель учится предсказывать метки. В обучении без учителя разметки нет, и модель ищет структуру в данных (кластеры, аномалии, снижение размерности). В обучении с подкреплением агент взаимодействует со средой и учится на наградах. В нашем курсе основной акцент — на обучении с учителем, а именно на классификации.

Обучение с учителем

Обучение с учителем делится на две основные задачи: регрессия и классификация. В регрессии целевая переменная непрерывна: например, прогноз цены квартиры, температуры, курса валюты. В классификации целевая переменная дискретна: например, спам/не спам, болен/здоров, выжил/не выжил. Мы рассматриваем классификацию, но многие принципы (переобучение, кросс-валидация, метрики) применимы и к регрессии.

Процесс обучения с учителем включает несколько этапов. Первый этап — сбор и подготовка данных. Второй этап — выбор модели. Третий этап — обучение модели на тренировочных данных. Четвёртый этап — оценка качества на валидационных данных. Пятый этап — настройка

гиперпараметров. Шестой этап — финальная оценка на тестовых данных. Седьмой этап — интерпретация и внедрение модели.

Обучение без учителя

Обучение без учителя используется, когда разметки нет. Основные задачи: кластеризация (группировка объектов по схожести), снижение размерности (сжатие признакового пространства), поиск аномалий (выявление нетипичных объектов). Кластеризация применяется в сегментации клиентов, группировке документов, анализе изображений. Снижение размерности применяется для визуализации и ускорения обучения. Поиск аномалий — для обнаружения мошенничества, сбоев, ошибок.

В нашем курсе обучение без учителя подробно не рассматривается, но важно понимать его отличие от обучения с учителем: здесь нет правильных ответов, и модель ищет структуру самостоятельно. Это делает оценку качества сложнее: нет метрик «правильно/неправильно», есть только внутренние метрики (например, компактность кластеров).

Переобучение и недообучение

Одна из центральных проблем машинного обучения — переобучение (overfitting) и недообучение (underfitting). Переобучение — это ситуация, когда модель слишком хорошо запомнила обучающие данные, включая шум и случайные fluctuations, и плохо работает на новых данных. Недообучение — это ситуация, когда модель слишком простая и не улавливает закономерности даже на обучающих данных.

Причины переобучения: слишком сложная модель, мало данных, много признаков, долгое обучение. Причины недообучения: слишком простая модель, мало признаков, недостаточное обучение. Компромисс между переобучением и недообучением называется смещением и дисперсией (bias-

variance tradeoff). Смещение — это ошибка, связанная с упрощением модели. Дисперсия — это ошибка, связанная с чувствительностью модели к данным. Идеальная модель имеет низкое смещение и низкую дисперсию.

Способы борьбы с переобучением: упрощение модели, регуляризация, увеличение данных, кросс-валидация, ранняя остановка, ансамбли. Способы борьбы с недообучением: усложнение модели, добавление признаков, увеличение времени обучения.

```
from sklearn.model_selection import learning_curve
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')  # скрываем ConvergenceWarning

# Загрузка данных
data = load_breast_cancer()
X, y = data.data, data.target

# Разделение данных
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Модель с масштабированием
model = Pipeline([
```

```
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression(max_iter=1000))
])

# Кривая обучения
train_sizes, train_scores, val_scores = learning_curve(
    model, X_train, y_train, cv=5, scoring='accuracy',
    train_sizes=np.linspace(0.1, 1.0, 10)
)

plt.plot(train_sizes, train_scores.mean(axis=1), label='Обучение')
plt.plot(train_sizes, val_scores.mean(axis=1), label='Валидация')
plt.xlabel('Размер обучающей выборки')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Кривая обучения')
plt.show()
```

Разделение данных

Для оценки обобщающей способности модели данные разделяют на несколько частей. Обучающая выборка (train) используется для обучения модели. Валидационная выборка (validation) — для настройки гиперпараметров и выбора модели. Тестовая выборка (test) — для финальной оценки. Обычно данные делят в пропорции 60/20/20 или 70/15/15.

Стратификация — это метод разделения, при котором сохраняются пропорции классов. Это важно при дисбалансе классов: если в данных 90% класса A и 10% класса B, то в каждой выборке должно быть примерно такое же соотношение. Стратификация обеспечивается параметром stratify в функции train_test_split.

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.4, random_state=42, stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp
)
```

Почему нельзя оценивать модель на обучающих данных? Потому что модель может просто запомнить их и показать высокую точность, но на новых данных работать плохо. Поэтому тестовая выборка должна быть независимой и использоваться только один раз — для финальной оценки. Если использовать её многократно для настройки, она «перестаёт быть тестовой» и модель начинает подстраиваться под неё.

Кросс-валидация

Кросс-валидация — это метод оценки модели, при котором данные разбиваются на k частей (фолдов). Модель обучается на k−1 частях и оценивается на оставшейся. Процесс повторяется k раз, каждый раз с новой валидационной частью. Результаты усредняются. Это даёт более надёжную оценку, чем однократное разделение, особенно на небольших данных.

Основные виды кросс-валидации: k-fold (стандартная), stratified k-fold (сохраняет пропорции классов), leave-one-out (каждый объект — отдельный фолд), time series split (для временных рядов). В sklearn кросс-валидация реализована функциями cross_val_score() и cross_validate().

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression(max_iter=1000)
scores = cross_val_score(model, X, y, cv=5, scoring='f1')

print(f'F1 по фолдам: {scores}')
print(f'Среднее F1: {scores.mean():.3f}')
print(f'Стандартное отклонение: {scores.std():.3f}')
```

Преимущества кросс-валидации: более надёжная оценка, использование всех данных для обучения и валидации, возможность оценить стабильность модели. Недостатки: больше времени на обучение, сложнее интерпретировать при большом числе фолдов.

Библиотека sklearn

Scikit-learn (sklearn) — основная библиотека Python для машинного обучения. Она предоставляет единообразный интерфейс для десятков алгоритмов: линейные модели, деревья, ансамбли, SVM, KNN, кластеризация, снижение размерности. sklearn интегрируется с NumPy и Pandas, что делает её удобной для работы с данными.

Основные модули sklearn: sklearn.model_selection (разделение данных, кросс-валидация, поиск гиперпараметров), sklearn.preprocessing (масштабирование, кодирование), sklearn.linear_model (линейные модели), sklearn.tree (деревья), sklearn.ensemble (ансамбли), sklearn.svm (метод опорных векторов), sklearn.metrics (метрики качества), sklearn.pipeline (конвейеры).

Основной интерфейс sklearn единообразен: у каждой модели есть методы fit() (обучение), predict() (предсказание), predict_proba() (вероятности), score() (оценка). Это делает код единообразным и понятным.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]
print(f'Accuracy: {model.score(X_test, y_test):.3f}')
```

Pipeline и ColumnTransformer

Pipeline — это инструмент sklearn, который объединяет несколько шагов обработки данных и обучения модели в один объект. Pipeline позволяет избежать утечки данных (data leakage), упрощает кросс-валидацию и делает код чище. Каждый шаг Pipeline — это кортеж (название, объект). Последний шаг — модель.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression(max_iter=1000))
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

ColumnTransformer — это инструмент, который позволяет применять разные преобразования к разным столбцам. Например, числовые признаки масштабировать, а категориальные — кодировать. ColumnTransformer принимает список кортежей (название, преобразователь, список столбцов).

```
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# 1. Загрузка данных
df = sns.load_dataset('titanic')
df = df.dropna(subset=['age', 'embarked'])

# 2. Формирование X и y
X = df[['age', 'fare', 'sex', 'embarked']]
y = df['survived']

# 3. Разделение данных
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 4. Определение признаков
numeric_features = ['age', 'fare']
categorical_features = ['sex', 'embarked']

# 5. ColumnTransformer
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(drop='first'), categorical_features)
])
```

```
# 6. Pipeline
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(max_iter=1000))
])

# 7. Обучение
pipeline.fit(X_train, y_train)

# 8. Оценка
print(f'Accuracy: {pipeline.score(X_test, y_test):.3f}')
```

Преимущества Pipeline: исключает утечку данных, упрощает кросс-валидацию, делает код воспроизводимым, позволяет сохранять и загружать модель целиком. Pipeline — это стандартный способ построения моделей в sklearn.

Постановка задачи классификации

Классификация — это задача отнесения объекта к одной из заранее определённых категорий. Формально: дано пространство признаков X и множество меток Y. Требуется найти функцию f: X → Y, которая для любого объекта x возвращает метку класса. В обучении с учителем есть размеченная выборка $D = \{(x^i, y^i)\}$, на которой обучается модель.

Типы задач классификации: бинарная (два класса), многоклассовая (более двух классов, каждый объект принадлежит ровно одному классу), многометочная (один объект может принадлежать нескольким классам одновременно). Примеры: спам/не спам (бинарная), распознавание цифр (многоклассовая), присвоение тегов статье (многометочная).

Основные алгоритмы классификации: логистическая регрессия, метод опорных векторов (SVM), деревья решений, случайный лес, градиентный бустинг, K-ближайших соседей, наивный Байес. Каждый алгоритм имеет

свои преимущества и недостатки. Выбор зависит от задачи, данных и требований к интерпретируемости.

Выводы по лекции

В ходе лекции мы рассмотрели основы машинного обучения: постановку задач, типы обучения, переобучение и недообучение, разделение данных, кросс-валидацию. Мы изучили библиотеку sklearn, Pipeline и ColumnTransformer. Мы рассмотрели постановку задачи классификации и основные алгоритмы. Ключевая идея лекции состоит в том, что машинное обучение — это не просто применение алгоритма, а целостный процесс, включающий подготовку данных, выбор модели, обучение, оценку и настройку.

Контрольные вопросы:

1. Что такое машинное обучение и чем оно отличается от традиционного программирования?
2. Перечислите и кратко опишите основные типы машинного обучения: с учителем и без учителя.
3. Что такое переобучение (overfitting) и недообучение (underfitting)? Как их выявить и предотвратить?
4. Зачем данные разделяют на обучающую, валидационную и тестовую выборки?
5. Что такое кросс-валидация и для чего она используется?

Список источников:

1. Кугаевских, А. В. Классические методы машинного обучения : учебное пособие / А. В. Кугаевских, Д. И. Муромцев, О. В. Кирсанова. — Санкт-Петербург : Университет ИТМО, 2022. — 53 с.

2. Рашка, С. Машинное обучение с PyTorch и Scikit-Learn : книга / С. Рашка, Ю. Лю, В. Мирджалили. — Астана : Флориант, 2024. — 688 с.

3. Плас, Дж. Вандер. Python для сложных задач: наука о данных и машинное обучение. — Санкт-Петербург : Питер, 2022. — 576 с.

4. Грас, Дж. Data Science. Наука о данных с нуля. — Санкт-Петербург : БХВ-Петербург, 2021. — 336 с.